CS 101 - Algorithms & Programming I

Spring 2026 - Lab 5

Due: Week of March 9

*Remember the **honor code** for your programming assignments.*
*For all labs, your solutions must conform to the CS101 style **guidelines**!*
All data and results should be stored in variables (or constants where appropriate) with meaningful names.

The objective of this lab is to practice designing and implementing static methods. As always, analyzing the problem and outlining your approach on paper before starting implementation is strongly recommended.

For the methods described below, you must not use built-in Java methods that directly perform the requested tasks (e.g., `String.toUpperCase()`, `StringBuilder.reverse()`, `Character.isLetter()`, etc.), unless explicitly permitted. You may rely on ASCII values and tables when implementing character-based logic.

0. Setup Workspace

Open VS Code and load your previously created workspace named labs_ws.

- Inside the labs directory, create a new folder named `lab5`.

In this lab, you are expected to create two Java classes/files inside labs/lab5 as described below. A third file for revisions will also be placed in this folder later. You must submit all required files separately without compressing them. Do not include files from previous labs in your submission.

In this lab, you will first implement several static helper methods and then use them to build programs that perform different text processing tasks based on user input.

1. String Processing Toolkit

Create a new file named `Lab05_Q1.java` inside the lab5 folder. In this class you will implement a collection of static methods used for analyzing and transforming textual data. You will also create helper methods that simplify the implementation of larger tasks.

Helper Methods

Before implementing the main functionality, develop the following reusable helper methods.

- `static char toUpper(char ch)`

  This method converts a lowercase letter to its uppercase equivalent. If the character is not lowercase, return it unchanged.

  Hint: Uppercase and lowercase letters appear in sequential ranges in the ASCII table. The difference between corresponding lowercase and uppercase letters is constant and can be computed using `'a' - 'A'`.

- `static boolean isAlphabetic(char ch)`

  Return true if the character is an alphabet letter (either uppercase or lowercase). Otherwise, return false.

  Hint: Alphabet characters occupy continuous ranges in the ASCII table.

- `static boolean isNumeric(char ch)`

  Return `true` if the character is a digit between `'0'` and `'9'`, otherwise return `false`.

- `static boolean isSeparator(char ch)`

  Return true if the character represents whitespace, including:

  - space `' '`
  - tab `'\t'`
  - newline `'\n'`

  Return false otherwise.

String Analysis

Implement the following methods to examine properties of strings.

- `public static boolean isMirrorText(String str)`

  A mirror text is a sequence that reads the same from left to right and right to left. Your method must perform a case-insensitive comparison while ignoring only separator characters.

  Example: "Step on no pets" is a mirror text.

- `public static boolean haveSameLetters(String str1, String str2)`

  Two strings are considered equivalent if they contain exactly the same letters in different orders. The comparison must be case-insensitive and should consider letters only.

Example: "Dormitory" and "Dirty room" are equivalent.

Hint: One possible approach is to scan characters from one string and remove matching occurrences from the other string. You may use `String.indexOf()` and `String.substring()` for this purpose after filtering non-letter characters.

- `public static int tokenCount(String str)`

  Return the number of tokens in the given string. A token is defined as any sequence of non-separator characters separated by whitespace.

  Example: "Welcome to CS101!" contains 3 tokens.

  Be careful to handle inputs containing a single token without whitespace.

String Transformation

- `public static String generateTag(String str)`

  A tag is a simplified string suitable for identifiers or URLs. Your method should:

  - convert the entire string to lowercase
  - replace one or more whitespace characters with a single underscore _
  - keep letters and digits only
  - remove all other characters
  - avoid leading or trailing underscores

  Example: "Java Programming 101!" → java_programming_101

  Hint: Use `StringBuilder` when constructing strings iteratively for better efficiency.

Menu-Driven Toolkit

The main method must present a menu that allows the user to select operations from the toolkit, enter input text, and view the results. The menu should repeat until the user chooses to exit.

```
=== String Toolkit ===
1) Mirror Text
2) Compare Letters
3) Token Count
4) Generate Tag
5) Exit
Select an option (1-5): 1
Enter text: Step on no pets
Result: true

=== String Toolkit ===
1) Mirror Text
2) Compare Letters
3) Token Count
4) Generate Tag
```

```
5) Exit
Select an option (1-5): 1
Enter text: Hello World
Result: false

=== String Toolkit ===
1) Mirror Text
2) Compare Letters
3) Token Count
4) Generate Tag
5) Exit
Select an option (1-5): 2
Enter first string: Dormitory
Enter second string: Dirty room
Result: true

=== String Toolkit ===
1) Mirror Text
2) Compare Letters
3) Token Count
4) Generate Tag
5) Exit
Select an option (1-5): 3
Enter text: Welcome to CS101!
Token count: 3

=== String Toolkit ===
1) Mirror Text
2) Compare Letters
3) Token Count
4) Generate Tag
5) Exit
Select an option (1-5): 4
Enter text: Java Programming 101!
Tag: java_programming_101

=== String Toolkit ===
1) Mirror Text
2) Compare Letters
3) Token Count
4) Generate Tag
5) Exit
Select an option (1-5): 5
Goodbye!
```

2. Encoding Toolkit

Create `Lab05_Q2.java` inside the lab5 folder. In this class, you will implement classical text transformation techniques using static methods.

Helper Methods

- `static boolean isLowerAlphabet(char ch)`

  Return true if the character is a lowercase letter.

  You may reuse the `isAlphabetic(char)` method from Part 1.

Shift Cipher

The shift cipher replaces each letter by moving it forward or backward in the alphabet by a fixed number of positions. The alphabet wraps around when the shift exceeds its bounds.

- `public static String shiftCipher(String text, int shift, boolean encrypt)`

  This method must:

  - shift alphabetic characters while preserving case
  - wrap around the alphabet correctly
  - leave digits, punctuation, and whitespace unchanged
  - encrypt when encrypt is true, decrypt otherwise

  Hint: Decryption can be implemented using a negative shift. Use the modulo operator `%` carefully to avoid negative results.

Reverse Alphabet Cipher

- `public static String reverseAlphabetCipher(String text)`

  This method replaces each letter with its opposite in the alphabet (A ↔ Z, B ↔ Y, etc.). The method must preserve letter case and leave non-alphabetic characters unchanged.

Text Reversal

- `public static String reverseString(String text)`

  Return a new string with characters in reverse order.

  Example: "Programming 101!" → "!101 gnimmargorP"

  Hint: `StringBuilder` is recommended for efficient reversal.

Menu for Encoding Toolkit

The main method should provide a menu that allows the user to test the cipher and reversal methods. The menu must repeat until the user selects the exit option.

```
=== Encoding Toolkit ===
1) Shift Cipher
2) Reverse Alphabet Cipher
3) Reverse Text
4) Exit
Choose an option (1-4): 1
Enter text: Hello World
Enter shift amount: 3
Type 'e' to encode or 'd' to decode: e
Result: Khoor Zruog

=== Encoding Toolkit ===
1) Shift Cipher
2) Reverse Alphabet Cipher
3) Reverse Text
4) Exit
Choose an option (1-4): 1
Enter text: Khoor Zruog
Enter shift amount: 3
Type 'e' to encode or 'd' to decode: d
Result: Hello World

=== Encoding Toolkit ===
1) Shift Cipher
2) Reverse Alphabet Cipher
3) Reverse Text
4) Exit
Choose an option (1-4): 2
Enter text: Abc XYZ
Result: Zyx CBA

=== Encoding Toolkit ===
1) Shift Cipher
2) Reverse Alphabet Cipher
3) Reverse Text
4) Exit
Choose an option (1-4): 3
Enter text: Programming 101
Reversed Text: 101 gnimmargorP

=== Encoding Toolkit ===
1) Shift Cipher
2) Reverse Alphabet Cipher
3) Reverse Text
4) Exit
Choose an option (1-4): 4
Goodbye!
```